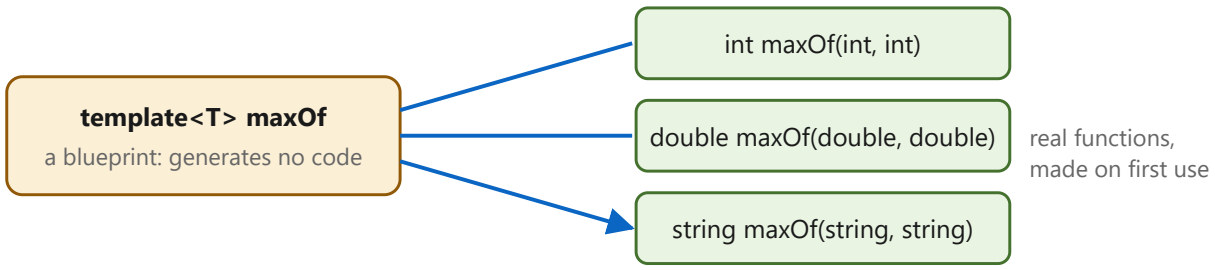
# Function Templates

One function, generated per type: instantiation, argument deduction, variadic templates and fold expressions, and forwarding references. Assumes Function Pointers and Functors.

## Why templates, and function templates




Compile-time polymorphism: no runtime cost, more code per type.

Without templates: `maxInt`, `maxDouble`, `maxString`... A **template** takes the type as a parameter:

```
template <typename T>
T maxOf(T a, T b) {
    return a < b ? b : a;
}

maxOf(3, 7);           // compiler deduces T = int
maxOf(2.5, 1.5);       // T = double
maxOf<double>(3, 2.5);// explicit: T = double, the int is converted
```

The template is not compiled itself. Each new type **instantiates** a real function: no runtime cost, more binary size, and errors appear only when a bad type is used.

KEY POINT Templates: **compile-time** (static) polymorphism, fastest, inlinable, more code, types known at compile time. Virtual: **runtime**, one code path, mixed objects, a vtable indirection.

## Template argument deduction

When you call a function template, the compiler works out `T` from the arguments. The rule depends on the **shape of the parameter**, and it is the same rule `auto` uses.

| PARAMETER | CALL | DEDUCED `T` | RULE |
|---|---|---|---|
| `T x` (by value) | `f(ci)` with `const int ci` | `int` | a copy: top-level `const` and references are **dropped** |
| `T x` (by value) | `f(arr)` with `int arr[3]` | `int*` | arrays and functions **decay** to pointers |
| `T& x` | `f(ci)` | `const int` | the reference keeps the argument's `const` |
| `T& x` | `f(arr)` | `int[3]` | an array keeps its size (usable to get its length) |
| `const T& x` | `f(5)` | `int` | binds temporaries; `const` is not part of `T` |
| `T&& x` | `f(i)` / `f(5)` | `int&` / `int` | forwarding reference (see Forwarding references) |

Compiled and checked with `static_assert`: by value, `const int` deduced `int` and the array deduced `int*`; by reference they deduced `const int&` and `int(&)[3]`.

- Deduction looks only at the **arguments**, never at the return type, and **no conversions** happen: `maxOf(3, 2.5)` fails because `T` would be both `int` and `double`.
- Default arguments and braced lists do not take part in deduction: `f({1, 2})` cannot deduce `T`. Name `T` explicitly: `f<std::vector<int>>({1, 2})`.
- Name the type yourself when needed: `maxOf<double>(3, 2.5)`.

> **WHEN DEDUCTION FAILS** The error says "couldn't deduce template parameter". Look for conflicting arguments, a braced list, or a parameter that does not mention `T` at all (e.g. only the return type does).

## Variadic templates and fold expressions

A **parameter pack** (`typename ... Args`) takes any number of arguments; `args ...` expands it. Before C++17: recursion with a base case.

```
template <typename T>
// base case: one argument
T sum(T v) { return v; }

template <typename T, typename ... Rest>
// peel one off, recurse
T sum(T first, Rest ... rest) { return first + sum(rest ...); }

sum(1, 2, 3);
```

The compiler instantiates a chain of real functions.

C++17 **fold expressions** replace the recursion:

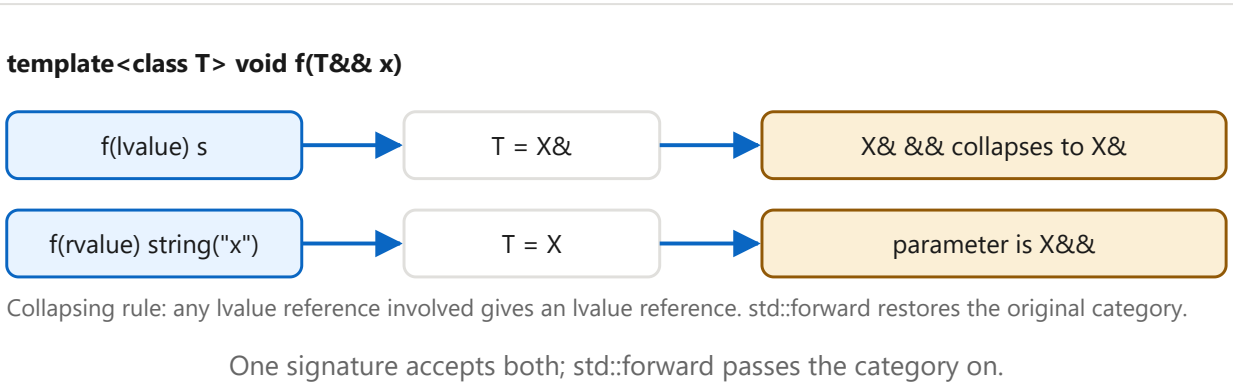
```
template <typename ... Args>
// unary right fold: a + (b + (c ...))
auto sum(Args ... args) { return (args + ...); }

template <typename ... Args>
// comma fold: do it for each arg
void print(Args&& ... args) { ((std::cout << args << ' '), ...); }

// number of elements in the pack
sizeof ...(Args);
```

Behind `make_unique`, `make_shared`, `emplace_back`, `tuple`, `thread`, `function`.

## Forwarding references




`T&&` **in a function template** is a **forwarding reference**, not an rvalue reference: it binds to anything and remembers lvalue vs rvalue.

### Is it really a forwarding reference?

Only when `T` is deduced for *that* function and the form is exactly `T&&` or `auto&&`:

| DECLARATION | FORWARDING REFERENCE? |
|---|---|
| `template<class T> void f(T&& x)` | yes: `T` is deduced for `f` |
| `auto&& x = expr;` | yes |
| `template<class T> void f(std::vector<T>&& v)` | no: not exactly `T&&`, so it binds rvalues only |
| `template<class T> void f(const T&& x)` | no: `const` disqualifies it |
| `template<class T> struct S { void f(T&& x); };` | no: `T` is fixed by the class, so nothing is deduced for `f` |
| `void f(Widget&& w)` | no: a plain rvalue reference |

### Reference collapsing

Reference to reference collapses: **any lvalue reference makes it an lvalue reference; only `&& + &&` stays rvalue.**

| WRITTEN | COLLAPSES TO |
| --- | --- |
| `T& &`, `T& &&`, `T&& &` | `T&` |
| `T&& &&` | `T&&` |

The last step, passing the value category on to another function with `std::forward`, needs `std::move` and rvalues first. It is the last section of the Move Semantics lesson.

---

## Pitfalls and good practice

| PITFALL | EXPLANATION / FIX |
| --- | --- |
| Huge unreadable error messages | The error is reported deep inside an instantiation. Read the *first* error and the line "required from here". Concepts or `static_assert` on traits give early, clear errors. |
| Template body in a `.cpp` file | Linker errors. Put the definition in the header. |
| `typename` for dependent types | Inside a template, `T::iterator` could be a type or a value. Write `typename T::iterator it;`. |
| Members of a dependent base class not found | Write `this->member` or `Base<T>::member`. |
| Code bloat | Every distinct `T` generates more machine code. Move type-independent logic into a non-template helper. |
| Deduction conflicts: `maxOf(3, 2.5)` | Two different deduced `T`s fail. Cast, specify `maxOf<double>`, or use two parameters. |
| `maxOf("a", "b")` with `const char*` | Compares pointer addresses, not text. Use `std::string` or `std::string_view`. |

## Further reading by section

Where each section of this lesson is explained in more depth. Pages are PDF viewer pages (the number your PDF reader shows).

| SECTION | BOOK | PART OF THE BOOK | PDF PAGES |
|---|---|---|---|
| **Why templates, and function templates** | C++ Primer | 16.1 Defining a template | 681 to 706 |
| | Programming: Principles and Practice | 18.1 to 18.3 Templates, generalizing Vector, range checking and exceptions | 527 to 541 |
| **Template argument deduction** | C++ Primer | 16.2 Template argument deduction | 707 to 722 |
| | C++ Primer | 16.3 Overloading and templates | 723 to 727 |
| | Effective Modern C++ | Item 1, Understand template type deduction | 27 to 36 |
| **Variadic templates and fold expressions** | C++ Primer | 16.4 Variadic templates | 728 to 734 |
| **Forwarding references** | C++ Primer | 16.2 Template argument deduction | 707 to 722 |
| | Effective Modern C++ | Item 24, Distinguish universal references from rvalue references | 182 to 186 |
| | Effective Modern C++ | Item 28, Understand reference collapsing | 215 to 221 |